



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Nulls in Apache Spark – Part 1

In any data processing system, a null represents the absence of a data item. It indicates that there is no valid data available for this field, or it is unknown.

And that's a big problem for data processing.

You do not know what to do with the null.

So it is critical that you learn the following things:

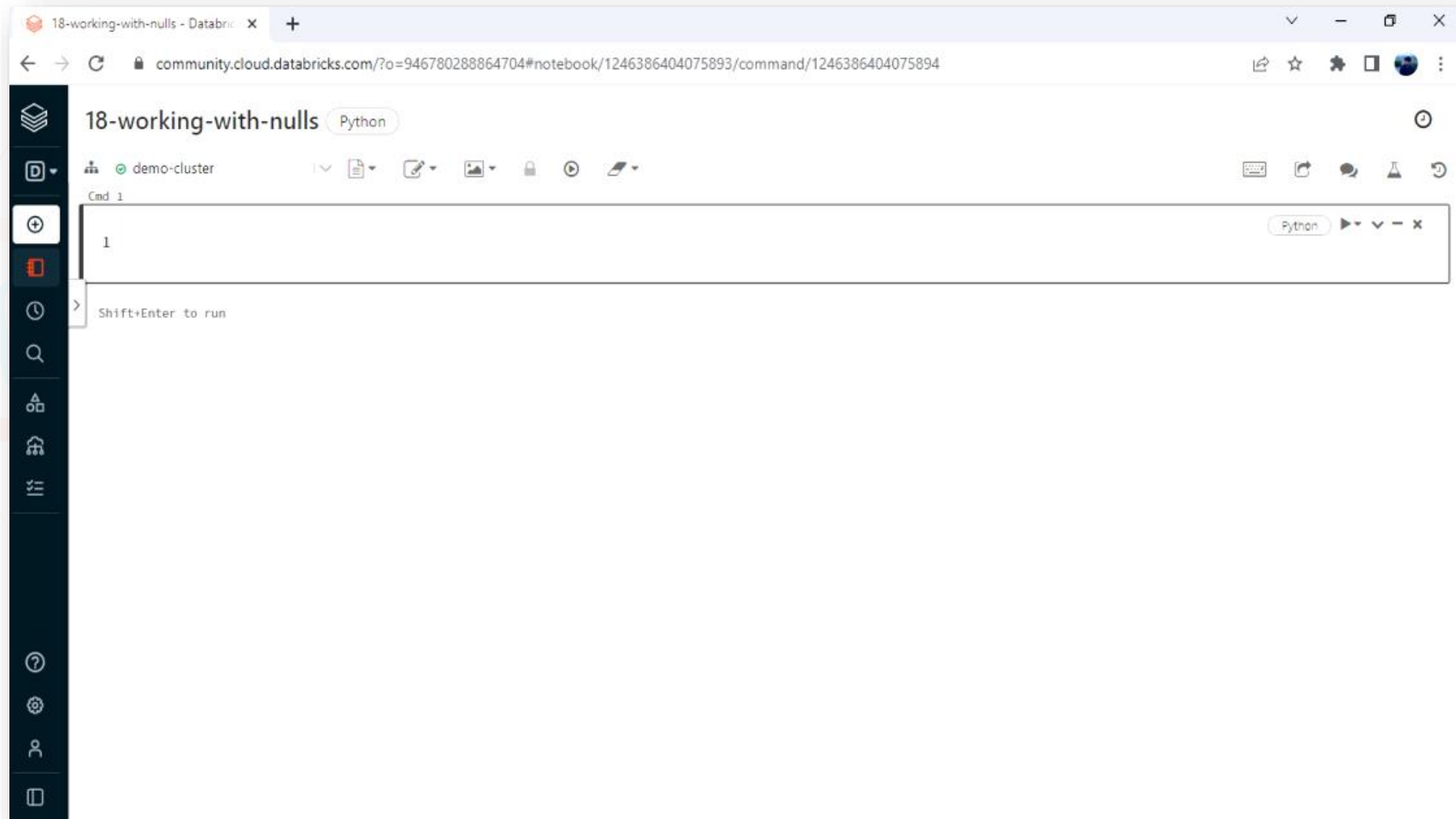
1. What are some common and complex null-related problems?
2. How Spark deals with Nulls?
3. How you can deal with Nulls?

Here are some of the common and complex null-related problems.

What are some common and complex Null problems?

1. Null in the Join Column
2. Null value Data Skew
3. Null in the Partition Column
4. Null in Spark Functions
5. Null in UDFs

Go to your Databricks workspace and create a new notebook.
(Reference : 18-working-with-nulls)



I am creating two Dataframes. The first one is an employee Dataframe, and the second data frame is for departments. The employee Dataframe has got some nulls in the department column. Now I want to join these two data frames using the department column.

18-working-with-nulls

Python

demo-cluster

Cmd 1

```
1 data_list1 = [(100, "Prashant", "Software"),
2               (101, "David", None),
3               (102, "Sushant", None),
4               (103, "Abdul", "Account"),
5               (104, "Shruti", "Software")]
6
7 data_list2 = [(501, "Software"),
8               (502, "Account")]
9
10 employee_df = spark.createDataFrame(data_list1).toDF("id", "first_name", "department")
11 department_df = spark.createDataFrame(data_list2).toDF("id", "department")
```

Command took 0.40 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:34:22 AM on demo-cluster

I am starting with the employee Dataframe and joining it with department_df using the department field. This expression uses default inner join. You can see the result highlighted below. You do not see any null records. The records for David and Sushant from the employee Dataframe are left aside. They didn't participate in the inner Join.

Cmd 2

```
1 employee_df.join(department_df, "department").select("first_name", "department").show()
```

▶ (3) Spark Jobs

first_name	department
Abdul	Account
Prashant	Software
Shruti	Software

Command took 3.62 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:35:22 AM on demo-cluster

The join operation in spark is a complex operation. It distributes data across the cluster over the network, and the distribution process is known as shuffle. But for now, you can assume that the shuffle is a time-consuming process. Most of the join operations in Spark suffers from performance problem, and the reason is a high volume of shuffle records.

In this example, we had two null records. They are not going to be part of the join results. But they will participate in the shuffle and slow down your join operation. The records with null values in the join column may not participate in the join operation. But they consume resources and slow down your join operation. If you eliminate these null records before the Join, you can tune your join operation to work faster.

18-working-with-nulls Python

demo-cluster

Cmd 1

```
1 data_list1 = [(100, "Prashant", "Software"),
2               (101, "David", None),
3               (102, "Sushant", None),
4               (103, "Abdul", "Account"),
5               (104, "Shruti", "Software")]
6
7 data_list2 = [(501, "Software"),
8               (502, "Account")]
9
10 employee_df = spark.createDataFrame(data_list1).toDF("id", "first_name", "department")
11 department_df = spark.createDataFrame(data_list2).toDF("id", "department")
```

Command took 0.40 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:34:22 AM on demo-cluster

Cmd 2

```
1 employee_df.join(department_df, "department").select("first_name", "department").show()
```

▶ (3) Spark Jobs

first_name	department
Abdul	Account
Prashant	Software
Shruti	Software

You have two tables, as shown below. You want to join these tables using a department.

But how does Join happen?

	id ▲	first_name ▲	department ▲
1	100	Prashant	Software
2	101	David	null
3	102	Sushant	null
4	103	Abdul	Account
5	104	Shruti	Software

	id ▲	department ▲
1	501	Software
2	502	Account

Spark will take both the tables and break them into Join key partitions. So the employee table will break into three partitions, as shown below. One partition for each unique join key.

We have three unique values in the employee table.

- 1. Software
- 2. Account
- 3. Null

So the employee table will create three partitions.

Similarly, the department table will create two partitions because we have two unique keys.

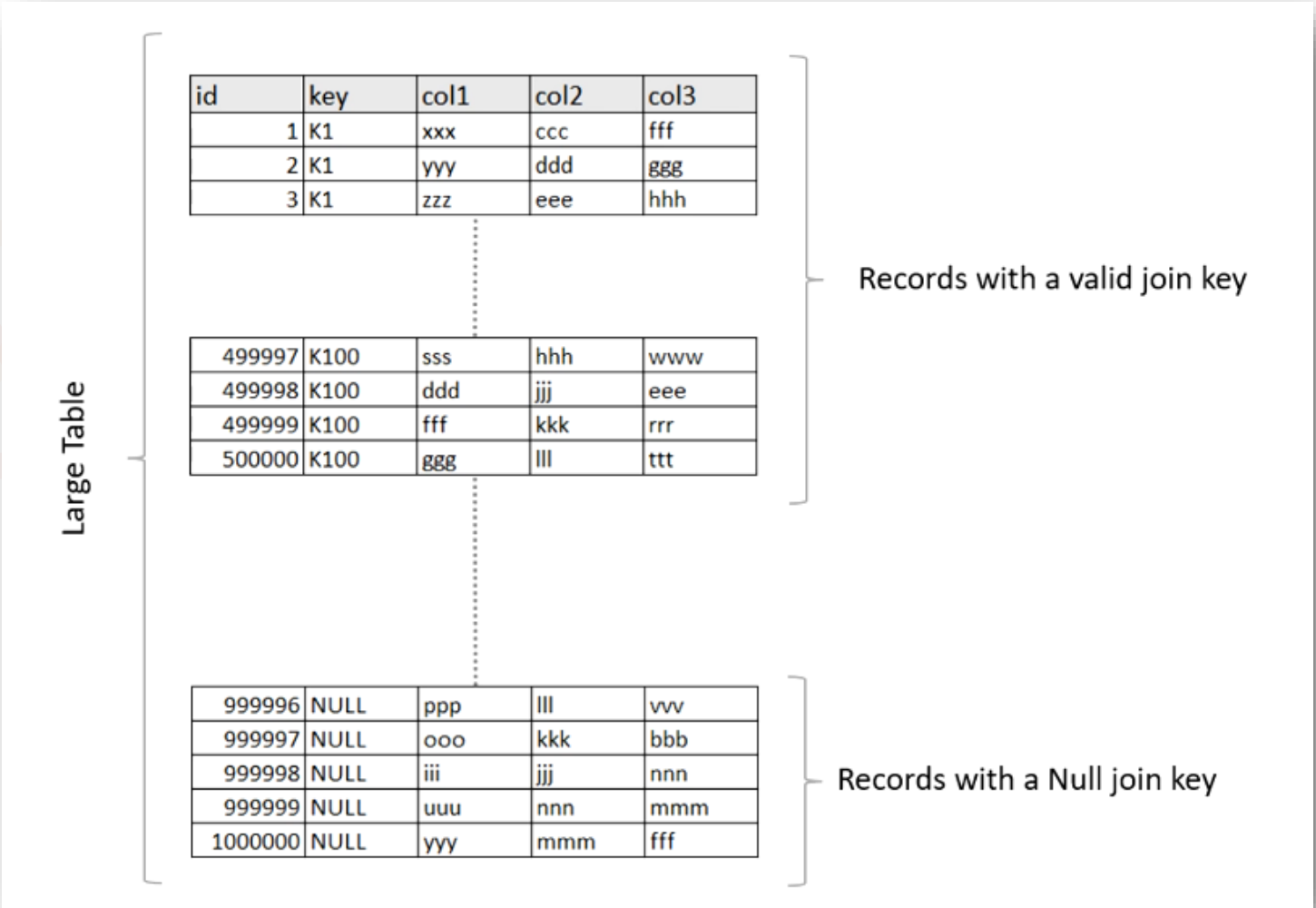
	id	first_name	department
1	100	Prashant	Software
5	104	Shruti	Software
4	103	Abdul	Account
2	101	David	null
3	102	Sushant	null

	id	department
1	501	Software
2	502	Account

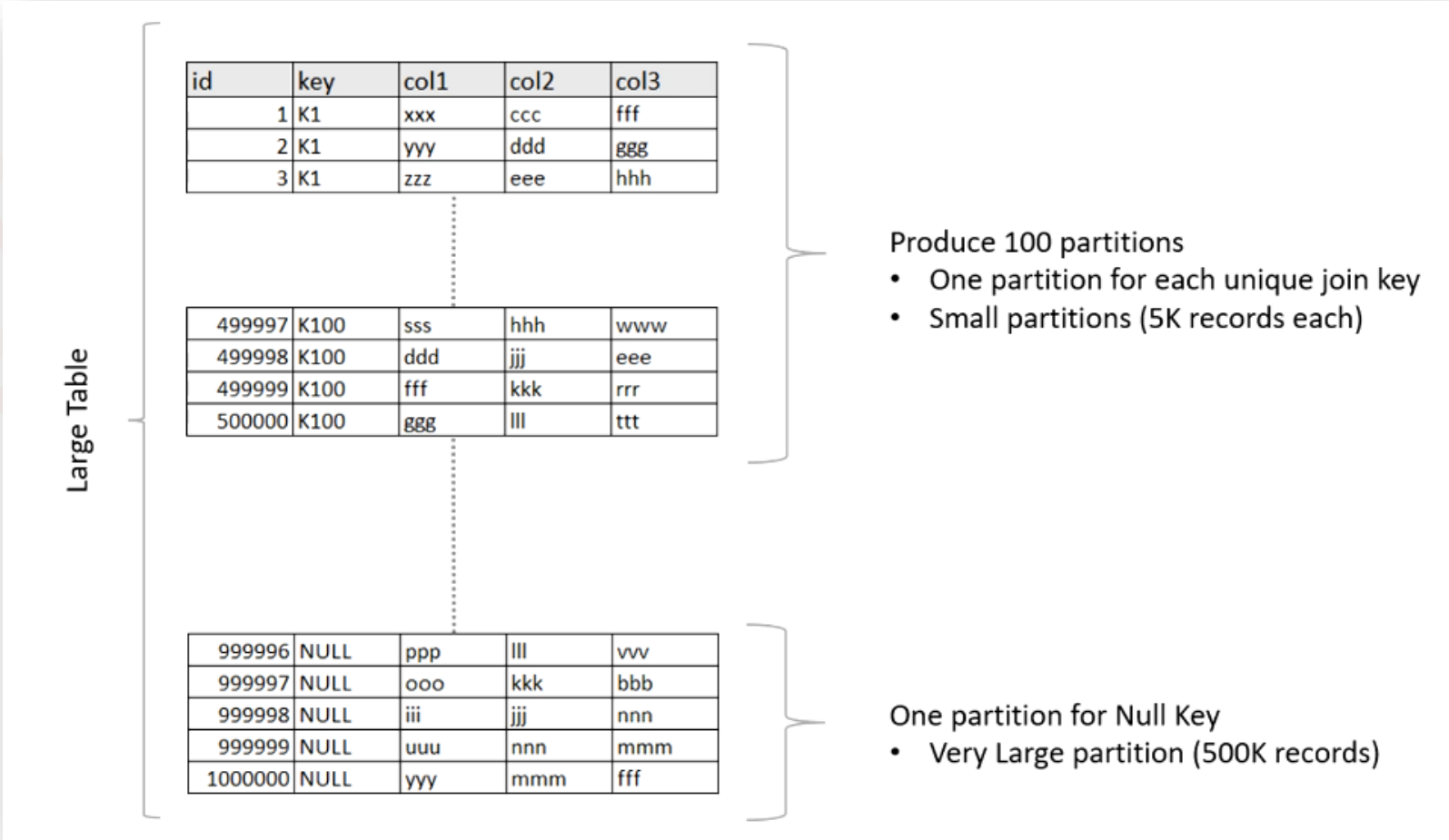
Now, the spark will combine partitions for the same key to apply the Join. So two employee records combine with the software department. The next employee record combines with the account department. But what about the null department? If you are doing an inner join, this one is left aside, and it does not combine with any partition. But this null partition still participates in the shuffle and consumes memory and CPU.



Now, let's scale the previous scenario. Assume you have a table of one million records. But you have only a hundred unique join keys. Each unique key has approximately 5K records, so you have half a million records with non-null join keys. The remaining half a million records are null.



If you try to join this table with another table using the join key. Spark will split this table into key-wise partitions. You will get 100 partitions for each unique key. And you will also get one partition for all the null values. These hundred partitions will have approximately 5K records in each partition. But the null partition will get 500K records.



This problem is called the data skew problem. Spark is distributed processing system.

It partitions your data on keys, and that happens in many scenarios.

Join operation is one of the most common partitioning scenario.

So whenever you are breaking your table into key-based partitions, you have a danger of creating a data skew.

In the data skew situation, all other partitions are of small size, but one or more partitions are disproportionate and very large compared to others.

These skewed partitions are massive performance problems in Spark.

Due to these skewed partitions, your application may hang or even fail with an out-of-memory error.

And the null values may easily cause skewed partitions.

Null values in the join key are a massive performance risk.

So you must be aware of this problem and take care of it in your code.

A similar problem happens when you want to write partitioned data. Suppose you want to create a spark table and partition it by a location. However, the location column is null for 30% of your records.

What will happen?

You will create a table with a null partition having 30% of the records in a single partition.

That's another data skew problem.

So, Null values are often a reason for data skew when partitioning is used on the null value column.

We also have other null problems.

Many machine-learning Spark APIs do not handle null values, and they throw an exception if they see nulls in the Dataframe column.

For example, VectorAssembler is used by data scientists for combining features into a single vector and applying logistic regression and decision trees.

However, the VectorAssembler throws an exception if you have nulls in the Dataframe.

We are not learning machine learning here. However, I wanted to point out that some Spark functions may not handle null columns and throw an exception. So you should know how to handle nulls.

Another common source of the null problem is associated with the user-defined functions.

Spark allows you to create user-defined functions, and you can use those functions in Dataframe and Spark SQL.

But when you code a user-defined function, you must ensure that your UDF handles null values appropriately.

I often see UDFs throwing exceptions when you pass null values to them.

Now let us see how Spark deals with Nulls.

Here is a Person Dataframe. We have some records where the age is null.

18-working-with-nulls Python

demo-cluster
Command took 3.62 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:35:22 AM on demo-cluster

Cmd 3

```
1 person_list = [(100, "Prashant", 30),
2               (101, "David", None),
3               (102, "Sushant", None),
4               (103, "Abdul", 45),
5               (104, "Shruti", 28)]
6
7 person_df = spark.createDataFrame(person_list).toDF("id", "name", "age")
8 display(person_df)
```

▶ (3) Spark Jobs

	id	name	age
1	100	Prashant	30
2	101	David	null
3	102	Sushant	null
4	103	Abdul	45
5	104	Shruti	28

Showing all 5 rows.

Command took 0.91 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:44:52 AM on demo-cluster

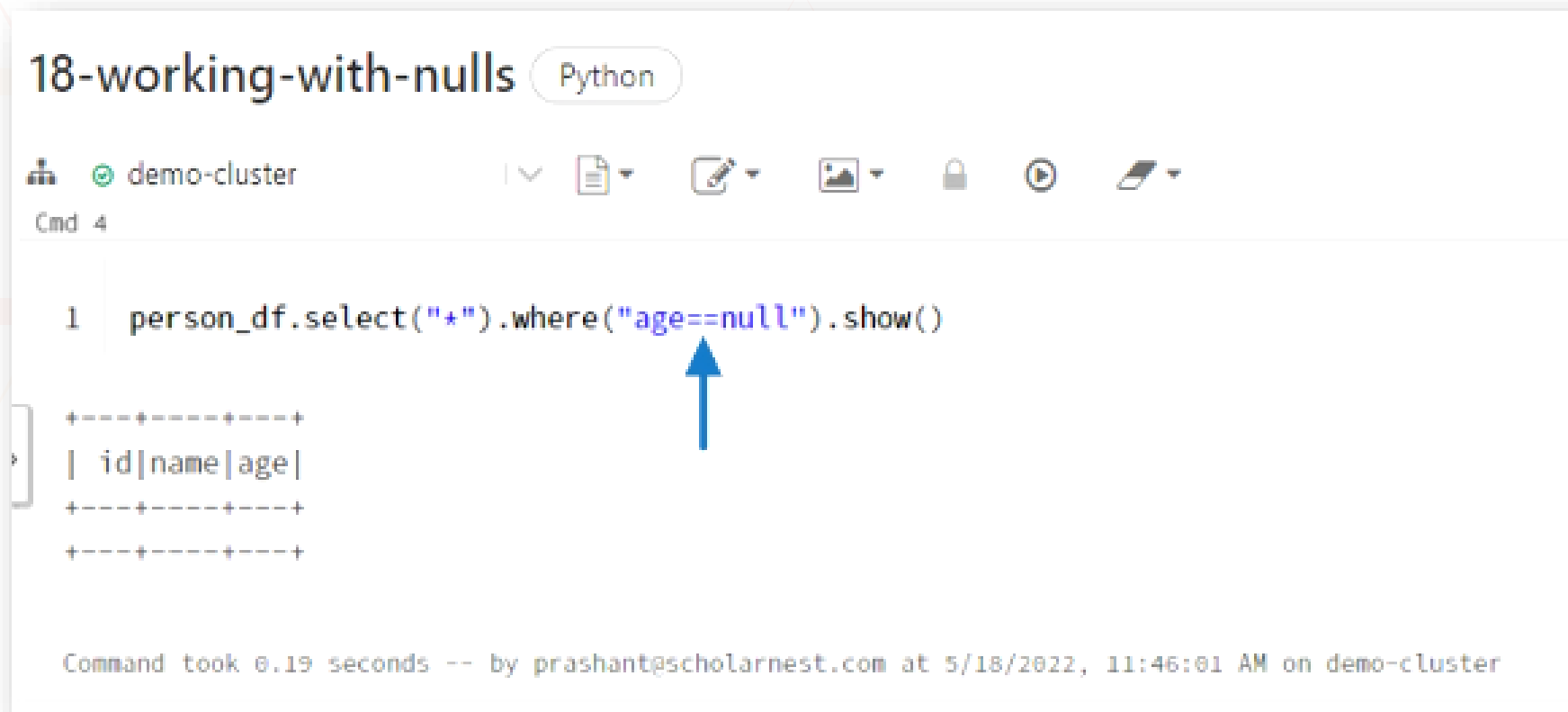
Now I want to tell you the following rules:

1. Null equality is tested using the "is null" operator
2. Comparison operator on null value is also a null.
3. Logical AND operator on a null value is also a null
4. Spark aggregations ignore the null value records
5. `groupBy()` will group all the null values in one bucket

Let us see some examples to understand these rules.



I want to select all records where the age equals null. I tried using the code below and get the desired output, but it didn't work. I was expecting two records. But I didn't get anything. Why? Because null equality is tested using the "is null" operator.



18-working-with-nulls Python

demo-cluster

Cmd 4

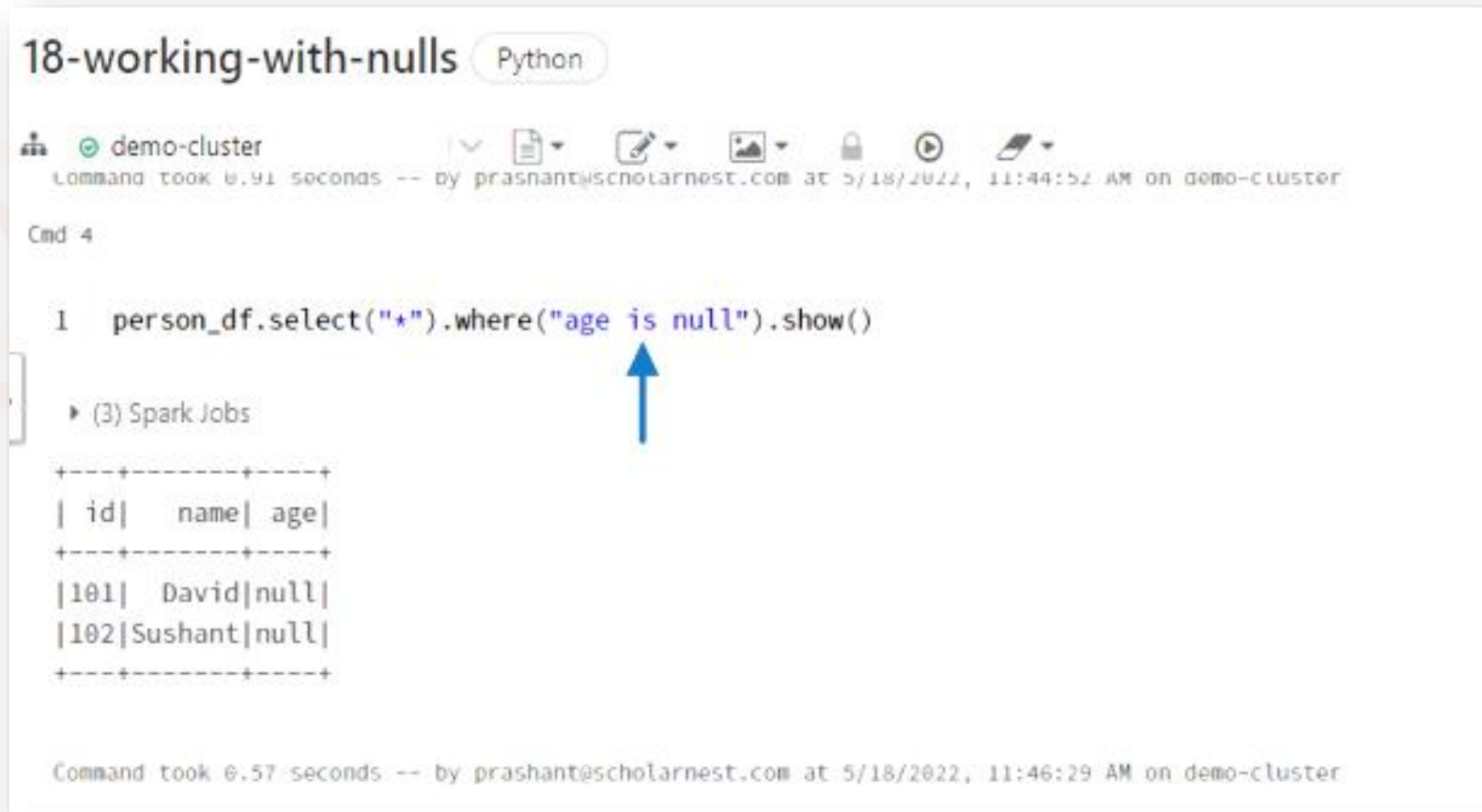
```
1 person_df.select("*").where("age==null").show()
```

id	name	age
----	------	-----

Command took 0.19 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:46:01 AM on demo-cluster

Now you see two records.

So remember to use the "is null" operator if you want to test something equal to null. The == operator doesn't work with nulls.



The screenshot shows a Databricks notebook interface. At the top, the title is "18-working-with-nulls" with a "Python" language selector. Below the title, it says "demo-cluster" and "Command took 0.91 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:44:52 AM on demo-cluster". The command input area shows "Cmd 4" and the code: `1 person_df.select("+").where("age is null").show()`. A blue arrow points to the text "age is null" in the code. Below the code, it says "(3) Spark Jobs". The output is a table with 3 columns: id, name, and age. The first two rows are visible: (101, David, null) and (102, Sushant, null). At the bottom, it says "Command took 0.57 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:46:29 AM on demo-cluster".

```
18-working-with-nulls Python
```

demo-cluster
Command took 0.91 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:44:52 AM on demo-cluster

Cmd 4

```
1 person_df.select("+").where("age is null").show()
```

(3) Spark Jobs

id	name	age
101	David	null
102	Sushant	null

Command took 0.57 seconds -- by prashant@scholarnest.com at 5/18/2022, 11:46:29 AM on demo-cluster

You can see it here that comparison operator on null value is also a null. I compared age > 29, and it returns true or false for non-null columns. But the result of a comparison is null for the null value column.

18-working-with-nulls Python

demo-cluster

Cmd 5

```
1 person_df.selectExpr("*", "age > 29").show()
```

(3) Spark Jobs

id	name	age	(age > 29)
100	Prashant	30	true
101	David	null	null
102	Sushant	null	null
103	Abdul	45	true
104	Shruti	28	false

Command took 1.11 seconds -- by prashant@scholarnest.com at 5/18/2022, 4:19:00 PM on demo-cluster

Now, we want to select all the records where the age is greater than 29. We have the following code to achieve this requirement. This expression will be true for only two records. So you will see only two records in the result. The other three records are not included because either the value is false or it is null.

Cmd 6

```
1 person_df.select("+").where("age > 29").show()
```

▶ (3) Spark Jobs

```
+---+-----+---+  
| id|   name|age|  
+---+-----+---+  
|100|Prashant| 30|  
|103|  Abdul| 45|  
+---+-----+---+
```

Command took 0.72 seconds -- by prashant@scholarnest.com at 5/18/2022, 4:19:42 PM on demo-cluster

Spark handles null values differently. Nulls are special cases in any expression, comparison, grouping, and aggregations. If you know SQL, you already know how databases handle nulls. Spark also deals with nulls in the same way.

I recommend that you go through the following documentation page once and check out some SQL examples of how spark treats nulls.
(Reference - <https://spark.apache.org/docs/latest/sql-ref-null-semantics.html>)



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com